جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

Lecture 4: Local Search

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture dives deep into the core of AI problem-solving, focusing specifically on **local search algorithms**. You'll uncover how AI agents navigate complex spaces to find optimal solutions using techniques such as **hill-climbing**, **simulated annealing**, and **genetic algorithms**. We will explore the strengths and weaknesses of each, equipping you with the fundamental skills to tackle real-world challenges through intelligent and adaptive search strategies.



Outline

1

Recap: Uniformed and Informed Search

Review of search algorithms that find paths from initial to goal states

2

Local Search Algorithms

Introduction to algorithms that optimize solutions locally

3

Hill-Climbing Search

Exploring greedy local search techniques and applications

4

Simulated Annealing

Temperature-based approach to escape local optima

5

Evolutionary Algorithms

Population-based methods inspired by biological evolution

6

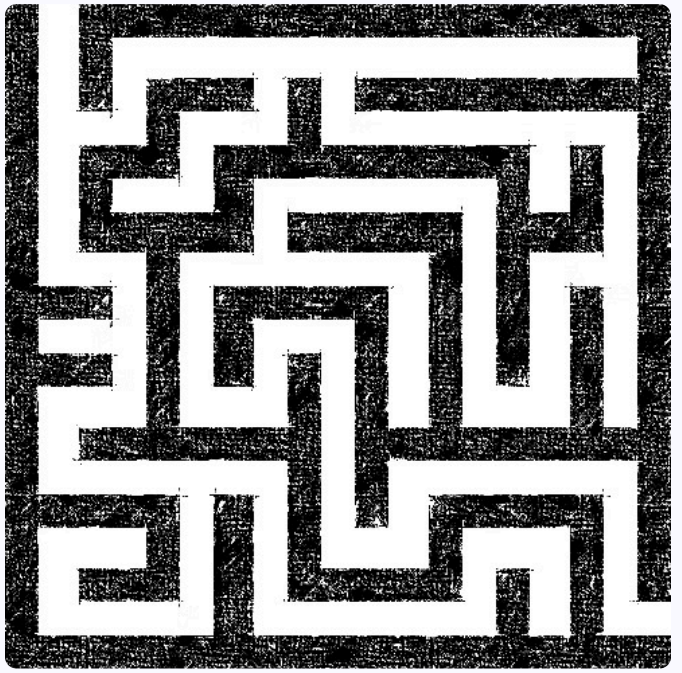
Search in Continuous Spaces

Techniques for optimizing functions in continuous domains

Recap: Uninformed and Informed Search

Characteristics:

- Plans the **best path** from a **given initial state** to a **given goal state**
  - Typically searches a **large portion of the search space**
  - Requires **significant time and memory** resources
  - Often provides **optimality** guarantees
  - Maintains a **search tree** to track paths
  - **Examples:** BFS, DFS, A*, Dijkstra's algorithm



Path finding from initial to goal state

These algorithms focus on **finding optimal paths** through **state spaces**, keeping **track of the entire search history** to ensure the best solution is found.

Local Search as an Optimization Problem

Each potential state is assigned a numerical value by an **objective, fitness, or utility function**. The search problem then transforms into:

$$\text{Find the state } s^* \text{ such that } f(s^*) \text{ is maximized (or minimized).}$$

Why Local Search?

Traditional Search

Primarily finds **optimal paths** from a start to a goal state.

In algorithms like BFS or A, the goal is to find a specific **sequence of actions** that leads from an initial state to a desired goal state. The path itself is crucial.*

Becomes **inefficient** or impossible in **large or infinite state spaces** where the path is irrelevant.

Local Search

Focuses on **discovering good states (solutions)**, not the path to them.

*Conversely, in local search, the focus is solely on finding a **"good" state**–one that represents an optimal or near-optimal solution–without regard for the path taken to reach it.*

Ideal for **combinatorial problems** like **n-Queens** where the final configuration is the goal, **not how you got** there.



Local Search Algorithms

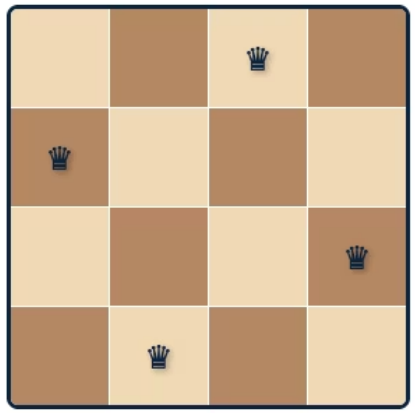
Key Concept:

Local search algorithms focus on **finding good solutions** by making **incremental improvements** through a series of **local moves** within the state space.

Characteristics:

- 🚀 **Start** with a current solution (state)
- 🔄 **Improve** by moving to a *better neighboring state*
- 🧠 Only keep track of the **current state** (no full history)
- 💾 **Memory-efficient** → uses very little storage
- 🎯 Well-suited for **optimization problems**

Current State



Possible Next States



Instead of searching for paths, local search algorithms **optimize a Utility/Objective function** by exploring the immediate neighborhood of the current state.

Local search is a bit like DFS that keeps going forward in one direction, but it never backtracks – it only focuses on the current state and its neighbors.

Local Search Algorithms: Key Features

Characteristics & Limitations

Local search algorithms operate by:

- Searching from a **start state** to **neighboring states**.
- **Not keeping track** of search paths or visited states.

⚠ This means they are **not systematic** and might never explore portions of the search space where an optimal solution resides.

Key Advantages & Applications

Despite their non-systematic nature, they offer significant benefits:

- **Memory Efficiency:** They use very little memory, only storing the current state.
- **Scalability:** Can find reasonable solutions in large or infinite state spaces where systematic algorithms are unsuitable.

They excel at solving **optimization problems**, aiming to find the **best possible state** according to a defined **objective function**.



Local Search Algorithms: Key Differences

Goal State Unknown

Focus is on **state utility/value**, not a predefined target goal state.

Path Not Important

Only the **optimal or near-optimal solution** state matters, not the path taken to reach it.

No Search Tree

Only the current state is stored, moving to a better state for memory efficiency in large problems.

These differences make **local search** ideal for **optimization problems** focused on finding good solutions.

n-Queens Problem: Defining the Search

1

State Space

Encompasses all possible arrangements of `n` queens on an `n x n` board. For instance, the **4-queens** problem has **1,820 unique** configurations.

2

State Representation

A **structured encoding** of queen positions, such as `(A2, B3, C1, D4)`,
where each element denotes a queen's column and row.

3

Objective Function

A **utility function** designed to **minimize** pairwise conflicts (queens attacking each other).

The goal is to **maximize utility** by achieving zero conflicts.

4

Local Neighborhood

Defined by states accessible through a **single, incremental change** from the current configuration, such as moving just one queen to a different square.

aniskoubaa.org

N-Queens Problem & Utility Functions – SE444

The N-Queens problem is a classic constraint satisfaction and optimization problem. The goal is to place n queens on an n×n chessboard such that no two queens can attack each other.

n-Queens Problem: A Classic Example

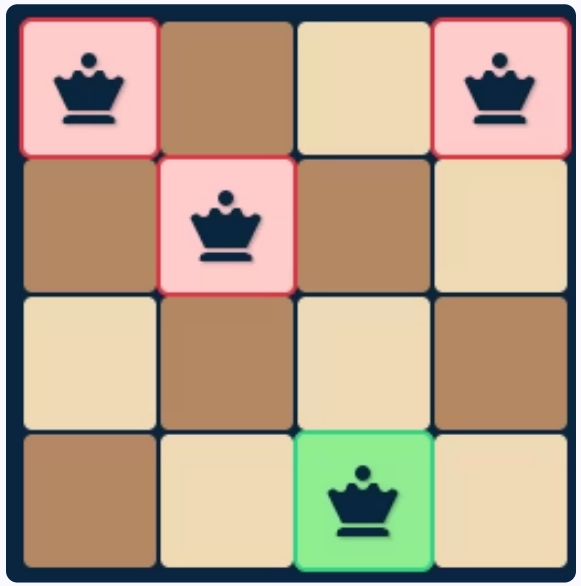
Problem Definition

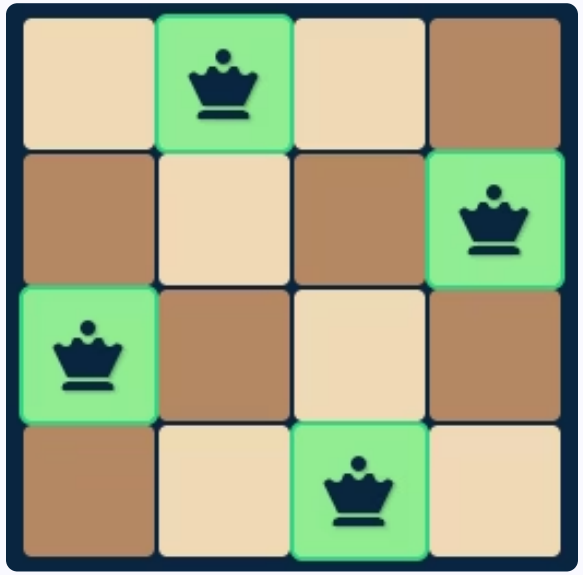
Place **n queens** on an **n × n chessboard** with no two queens on the same row, column, or diagonal.

Search Problem Components

- **State space:** All possible n-queen configurations (4.4 Billions)
- **State representation:** Queen positions on the board
- **Objective function:** **Minimize number of conflicts**
- **Local neighborhood:** States reachable by moving one queen

Different states in the n-Queens problem

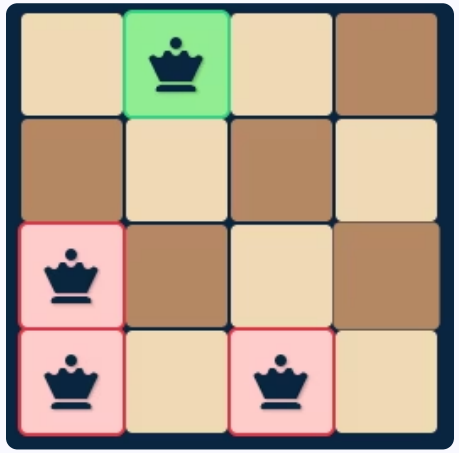




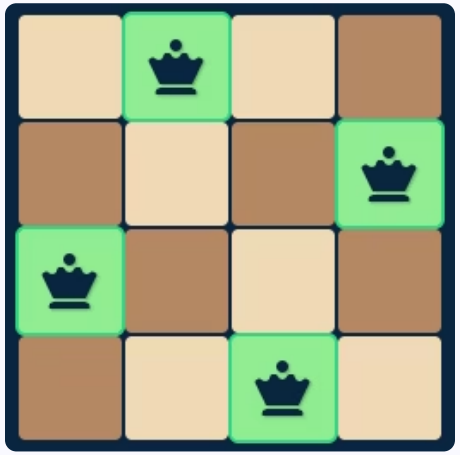
n-Queens Problem: A Classic Example

- **Utility Function:** In the n-Queens problem, utility is defined as the **negative number of conflicts** between queens.
- **Objective:** Maximize utility by reducing conflicts.
- The best solution has **zero conflicts**, which corresponds to a **utility of 0**.
- **What is a Conflict?** A conflict occurs when two queens share:
  - the same **row**
  - the same **column**
  - the same **diagonal**
- **Simplification:** one queen per column → only row & diagonal conflicts remain
  - State space is reduced from **1820** to **4^4=256**

2 conflicts = utility of -2



0 conflicts = utility of 0



n-Queens Problem: Utility Function

Utility Function

The goal is to place `n` queens on an `n x n` chessboard without any two queens attacking each other.

A `conflict` occurs when two queens share the same row, column, or diagonal.

We define the utility function (U) for a state `s` as:

$$U(s) = -(\text{number of conflicts in state } s)$$

What Does "Utility of 0" Mean?

- If there are **conflicts**, then `U(s) < 0`.
- For example, 2 conflicts result in `U(s) = -2`.
- If there are **no conflicts**, then `U(s) = 0`.

In this formulation, the **maximum possible utility is 0**, which corresponds to a valid, conflict-free solution for the n-Queens problem.

Why Not Positive Utilities?

Utility functions are `relative scales` and can be defined in various ways. While we could use alternative definitions that yield positive utilities for optimal solutions, the key principle remains the same:

- **Higher utility always indicates a better state.**
- The absolute value (e.g., 0 vs. 100) is less important than the ordinal relationship.

N-Queens: The Power of Local Search

👉 In the n-Queens problem, the vast state space makes it essential to choose the right search paradigm – unlike path-finding algorithms, local search focuses on directly improving solutions without exploring full paths.

Traditional Search (e.g., BFS, DFS, A*)

- **Focus:** Finding the **optimal path** from start to goal.
- **State Space:** Explores an **exponentially** large number of states, often `n!` for `n` queens.
- **Memory:** Requires storing vast portions of the search tree or visited states, becoming **memory-intensive** for large `n`.
- **Guarantees:** Can guarantee an optimal solution (zero conflicts) and the path to it, but often at a **prohibitive computational cost**.
- **Complexity:** For `n=8`, the state space can be 4.4 billion states, making systematic search impractical.

Local Search (e.g., Hill-Climbing, Simulated Annealing)

- **Focus:** Finding a **good solution state** (minimum conflicts), not the path.
- **State Space:** Explores only the immediate neighborhood of the current state, making it efficient for large problems.
- **Memory:** Only the current state is stored, leading to **minimal memory usage.**
- **Guarantees:** Does not guarantee the global optimum (can get stuck in local optima), but often finds high-quality solutions quickly.
- **Efficiency:** Can solve problems with hundreds or thousands of queens much **faster** than traditional search by iteratively improving the board configuration.

Traveling Salesman Problem (TSP)

👉 The Traveling Salesman Problem 🧳 challenges us to visit each city 🏙️ exactly once, return home 🔁, and find the shortest path 📏 possible.

Problem Breakdown

- **Goal:** Find the shortest tour connecting all cities.
- **Constraints:** Each city visited once, return to starting city.
- **Complexity:** The state space is vast: `(N-1)!/2` possible tours, making exhaustive search impractical for even moderate N. This highlights the need for **local search** methods.

Utility Function

For TSP, the goal is to **minimize tour length**.

The utility function is defined as:

$$U = -\text{Tour Length}$$

- 🔼 **Higher Utility:** shorter tour (better solution ✅)
- 🔽 **Lower Utility:** longer tour (worse solution ❌)

👉 This way, we turn a **minimization problem** into a **maximization problem**, which fits naturally with optimization algorithms.

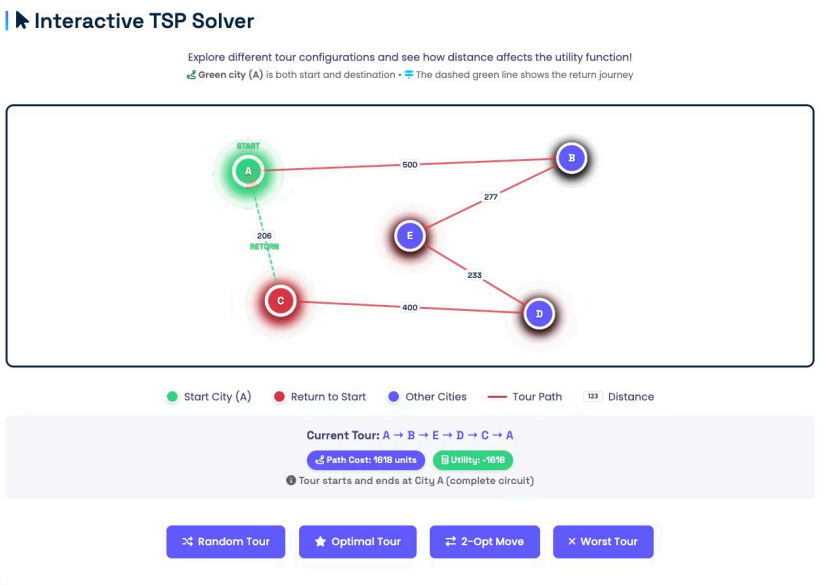
aniskoubaa.org

Traveling Salesman Problem & Utility Functions – SE444

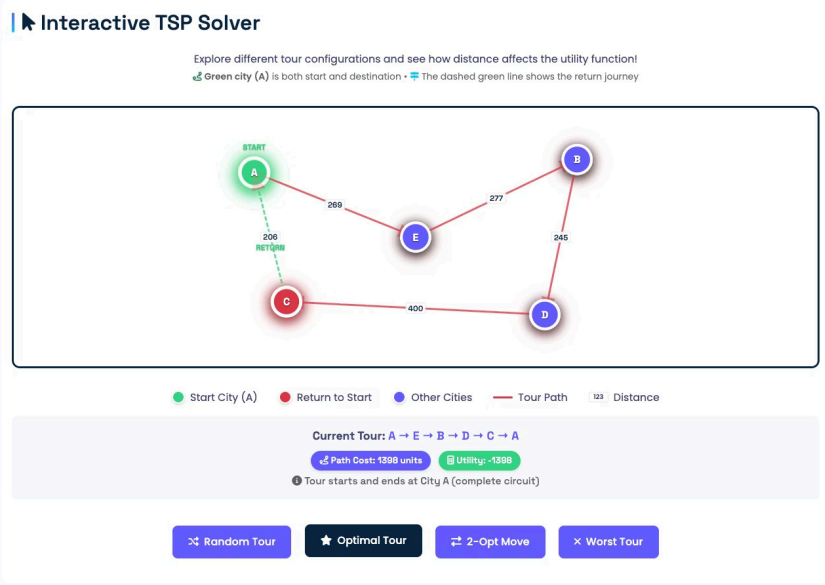
The Traveling Salesman Problem is a classic optimization problem where a salesman must visit n cities exactly once and return to the starting city, minimizing the total travel distance.

Traveling Salesman Problem (TSP)

Non-Optimal Path



Optimal Path



aniskoubaa.org

Traveling Salesman Problem & Utility Functions – SE444

The Traveling Salesman Problem is a classic optimization problem where a salesman must visit n cities exactly once and return to the starting city, minimizing the total travel distance.

Hill-Climbing Search



Hill-Climbing Search

Core Concept

Hill-climbing search (also called **greedy local search**) is a simple optimization algorithm that:

Maintains a Single Current State

Unlike other search algorithms that keep track of multiple states, hill-climbing only maintains information about the current state.

Evaluates Neighboring States

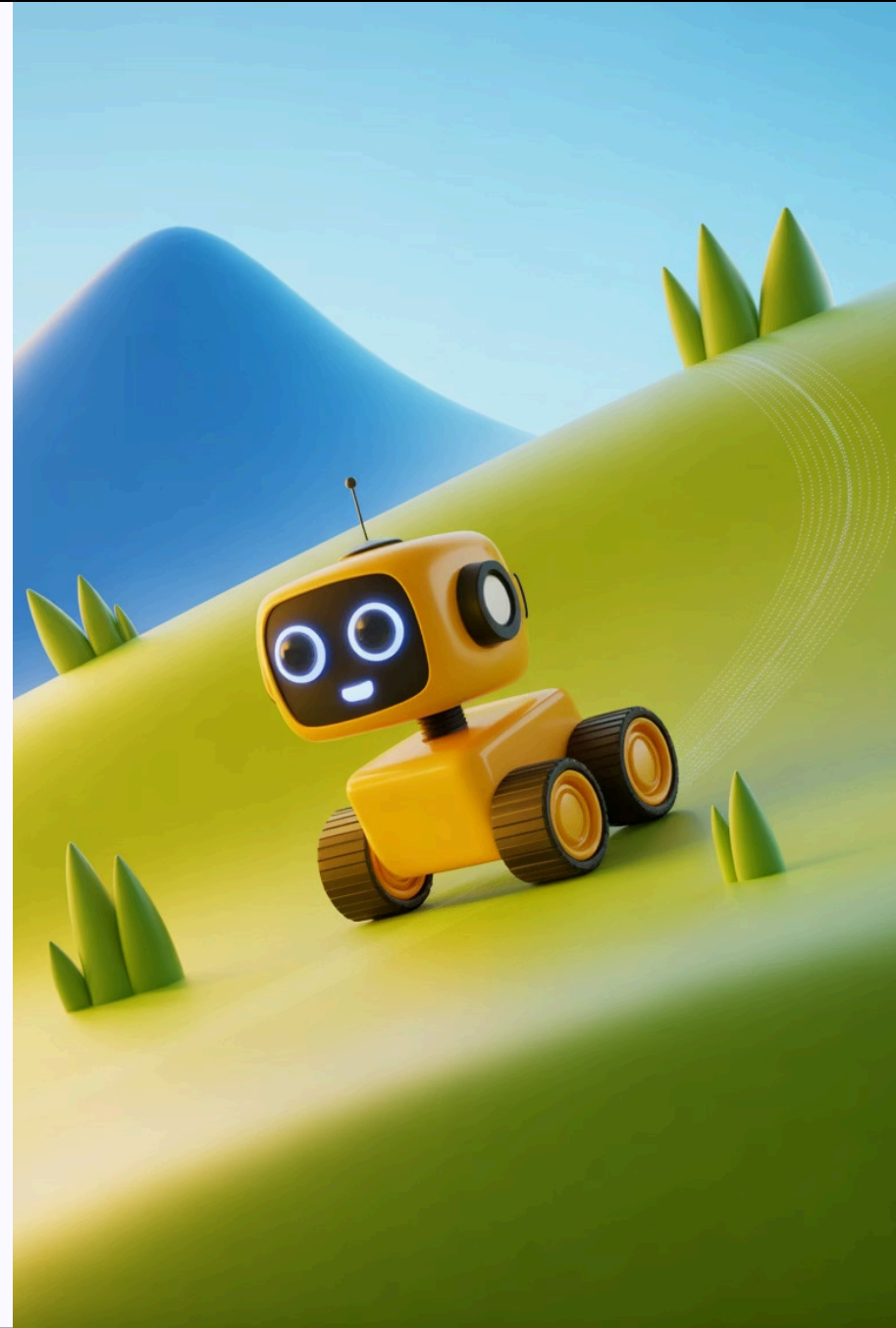
It examines the immediate neighbors of the current state to find better solutions.

Moves to Better States

It always moves to the **neighbor with the highest utility value** (steepest ascent) or to any better neighbor (first-choice).

Terminates at Local Optima

The algorithm **stops when no neighboring state has a higher utility** than the current state.



Hill Climbing vs. Greedy Search

Greedy Best-First Search (Informed Search) 🧭

- **Node Expansion:** Prioritizes expanding nodes with the lowest heuristic **h(n)** (estimated distance to goal).
- **Objective:** Aims to find a **complete path** from start to a goal state. 🗺️
- **Challenge:** Can get **trapped in local minima** if the heuristic function is inaccurate. 📉

Hill-Climbing (Local Search) ⛰️

- **State Management:** Maintains **only the current state**, discarding previous paths.
- **Movement:** Always moves to a neighboring state that offers a **higher utility** (improvement). 📈
- **Challenge:** Susceptible to getting stuck in **local maxima, plateaus, or ridges**, preventing global optimum. 🛑

Hill Climbing Algorithm

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor state of current
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor.

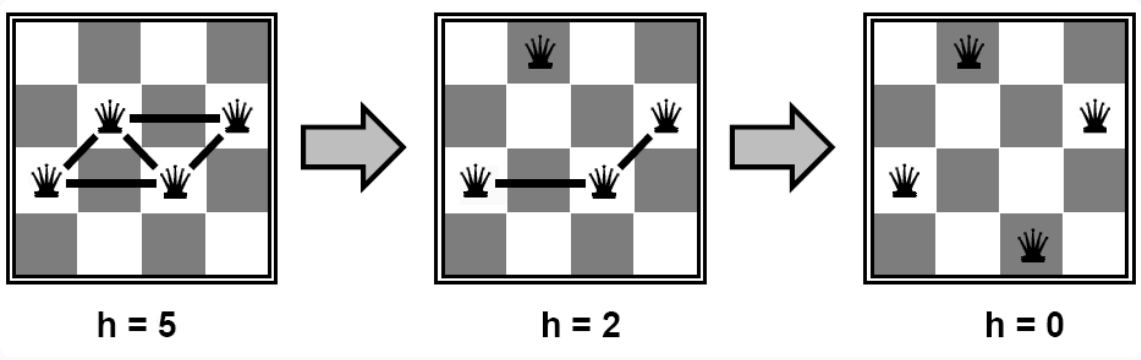
n-Queens Problem: Hill-Climbing Approach

Problem Formulation:

- **Goal:** Place n queens with no conflicts
- **State space:** All possible n-queen configurations
- **Constraint:** One queen per column
  - Reduces States from 1860 to 256 positions
- **State representation:** Row position of each queen in its column (e.g., 2, 3, 2, 3)
- **Objective function:** Minimize the number of conflicts
- **Local neighborhood:** Move one queen anywhere in its column

Improvement Strategy:

Find a local neighboring state by moving one queen within its column to reduce conflicts.



The constraint of one queen per column significantly reduces the search space while still allowing us to find solutions to the problem.

n-Queens Problem: State Representation

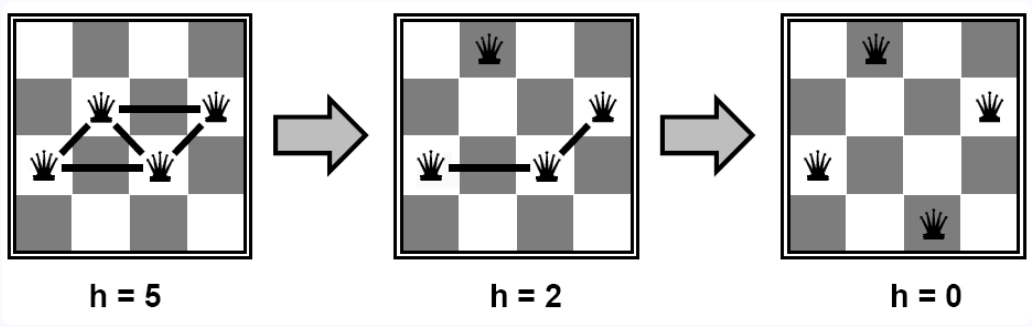
Efficient Representation:

Constrain the problem to **one queen per column** to **simplify state representation** & reduces search space.

Benefits:

- 🔽 Reduces state space from `n^n → n!`
- 🚫 Automatically eliminates column conflicts
- 📦 Enables compact representation as an array of length n
- ⚡ Makes neighbor generation more efficient

n-Queens board with one queen per column



Representation as array: [2,4,1,3]

This restriction makes the problem more tractable while still allowing us to find all possible solutions to the n-Queens problem.

Hill Climbing in n-Queens: Evaluating Neighbors

Finding the Best Move

To find the **best local move**, we must **evaluate all possible neighboring states** (i.e., calculate their objective function values).

1️⃣ For each **column**, try placing the queen in **every row**
2️⃣ Calculate the **number of conflicts** for each position ♟️⚔️
3️⃣ Select the position with the **minimum conflicts** ✅
4️⃣ If multiple positions tie, pick **one randomly** 🎲

This evaluation process is repeated until we reach a state where no single queen movement can reduce the number of conflicts further.



Objective values after moving the queens to different positions

The numbers in each square represent the objective function value (number of conflicts) if we move the queen to that position. Lower values are better.

[Hill Climbing N-Queens - Complete Algorithm Execution (4×4)]

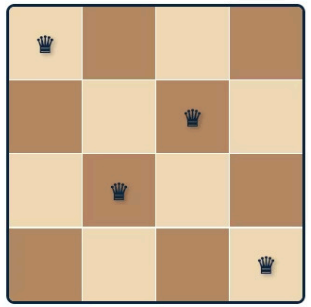
Hill Climbing may get stuck in non optimal solutions

Complete Neighbor Evaluation

Showing ALL possible neighbors - maximum 12 moves per step (4 queens × 3 alternate positions each)

U = -2 2 conflicts Move Q1 to row 3 Best Move	U = -2 2 conflicts Move Q2 to row 4	U = -2 2 conflicts Move Q3 to row 1	U = -2 2 conflicts Move Q4 to row 2
U = -3 3 conflicts Move Q1 to row 2	U = -3 3 conflicts Move Q2 to row 1	U = -3 3 conflicts Move Q3 to row 4	U = -3 3 conflicts Move Q4 to row 3
U = -4 4 conflicts Move Q1 to row 4	U = -4 4 conflicts Move Q2 to row 2	U = -4 4 conflicts Move Q3 to row 3	U = -4 4 conflicts Move Q4 to row 1

Step 5: Local Maximum Reached



Current board state

-2	2	5
Current Utility	Total Conflicts	Step Number

24
Neighbors Checked

Algorithm Status:

Best neighbor utility (-2) ≤ current utility (-2). Algorithm terminates at LOCAL MAXIMUM - cannot find solution even if one exists elsewhere!

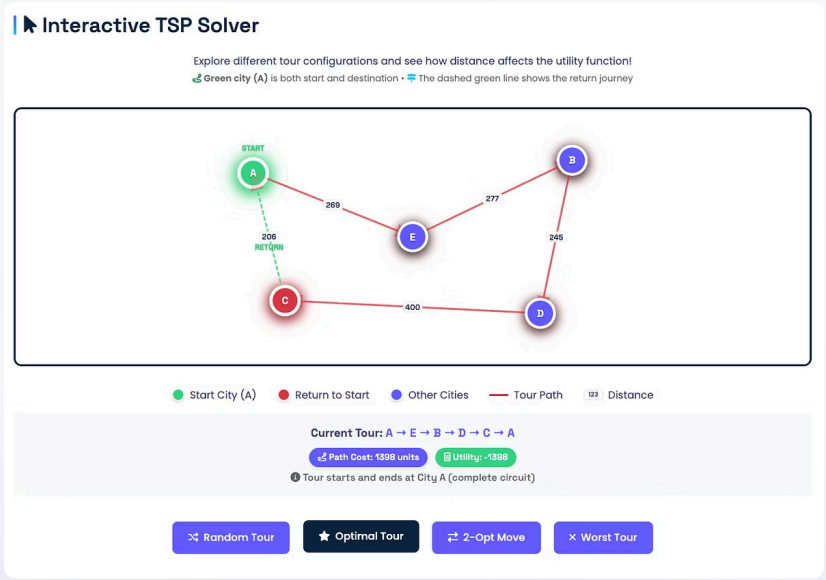
Traveling Salesman Problem (TSP)

Problem Definition:

Goal: Find the shortest tour connecting n cities, visiting each city exactly once and returning to the starting city.

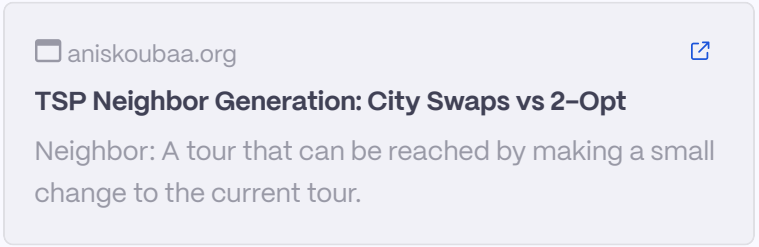
Search Formulation:

- **State space:** All possible tours (permutations of cities)
- **State representation:** Ordered sequence of cities
- **Objective function:** Total length of tour (to minimize)
- **Local neighborhood:** Tours reachable by reversing the order of a subsequence of cities



🔼 Key Aspects of Hill-Climbing for TSP Algorithm

- 🗺️ **State:** A candidate solution = a **tour**, represented as a permutation of cities.
- 🎯 **Objective:** Minimize the **total tour length** (shorter tours = better solutions).
- 🔄 **Neighbors:** Generated by making a **small change** to the tour.
  - Simplest: **swap two cities**.
  - Smarter: **2-opt** (reverse a segment of the tour).
- 🧠 **Move Rule:** Evaluate all neighbors.
  - Choose the one that **reduces the tour length the most**.
- 🛑 **Stopping Condition:** Stop when **no neighbor is better** → reached a **local optimum**.




```
HILL_CLIMB_TSP(cities, distance):
    tour = RANDOM_PERMUTATION(cities)
    improved = True
    while improved:
        improved = False
        current_best_length = CALCULATE_TOUR_LENGTH(tour, distance)
        best_neighbor_found = None
        for each possible swap (i, j) in tour:
            neighbor = SWAP(tour, i, j)
            neighbor_length = CALCULATE_TOUR_LENGTH(neighbor, distance)
            if neighbor_length < current_best_length:
                current_best_length = neighbor_length
                best_neighbor_found = neighbor
        if best_neighbor_found is not None:
            tour = best_neighbor_found
            improved = True
    return tour
```

Neighbor Generation in TSP

1. Swap Move

Select **two cities** in the tour and **swap** their positions. This creates a new neighboring tour.

- ✔️ **Simple & fast:** computationally inexpensive to generate.
- ❌ **Small changes:** may converge slowly and get stuck in local optima.

💡 **Small, local improvement** → good for **exploitation** (fine-tuning the current solution).

aniskoubaa.org

Loading...

2. 2-opt Move

Select two **non-adjacent edges** in the tour, remove them, and reconnect the remaining path in a different way by reversing a segment of the tour.

- ✔️ **Removes crossing paths:** can lead to significant improvements by untangling tours.
- ❌ **More computation:** generating and evaluating neighbors is more complex.

💡 Can produce **larger structural changes** → better for **exploration** (escaping poor local structures like crossings).

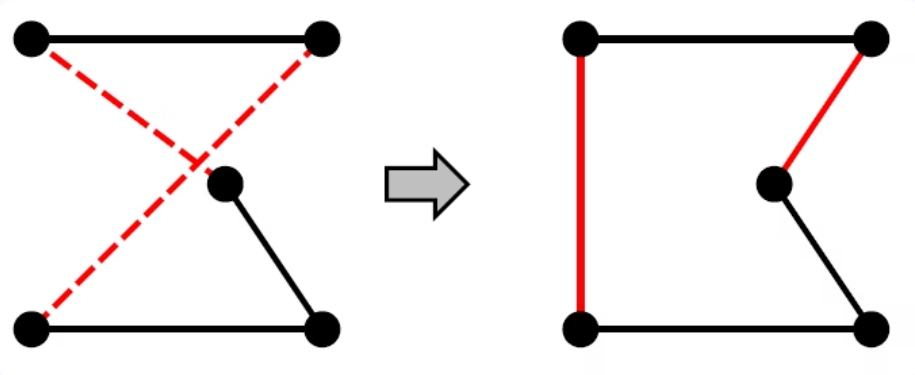
TSP: Improving Tours with 2-opt

🔹 Why 2-opt?

- ✂️ Replaces **two edges** in the tour
- ✅ Keeps all cities visited exactly once
- 🚫 Removes **crossing paths** (never optimal)
- ⚡ Makes small, efficient improvements

🔹 How it Works (Example C→E→D)

1️⃣ **Remove edges** (B - C) and (D - A)
2️⃣ **Add edges** (B - D) and (C - A)
3️⃣ **Check tour length**
4️⃣ **Accept** if the new tour is shorter

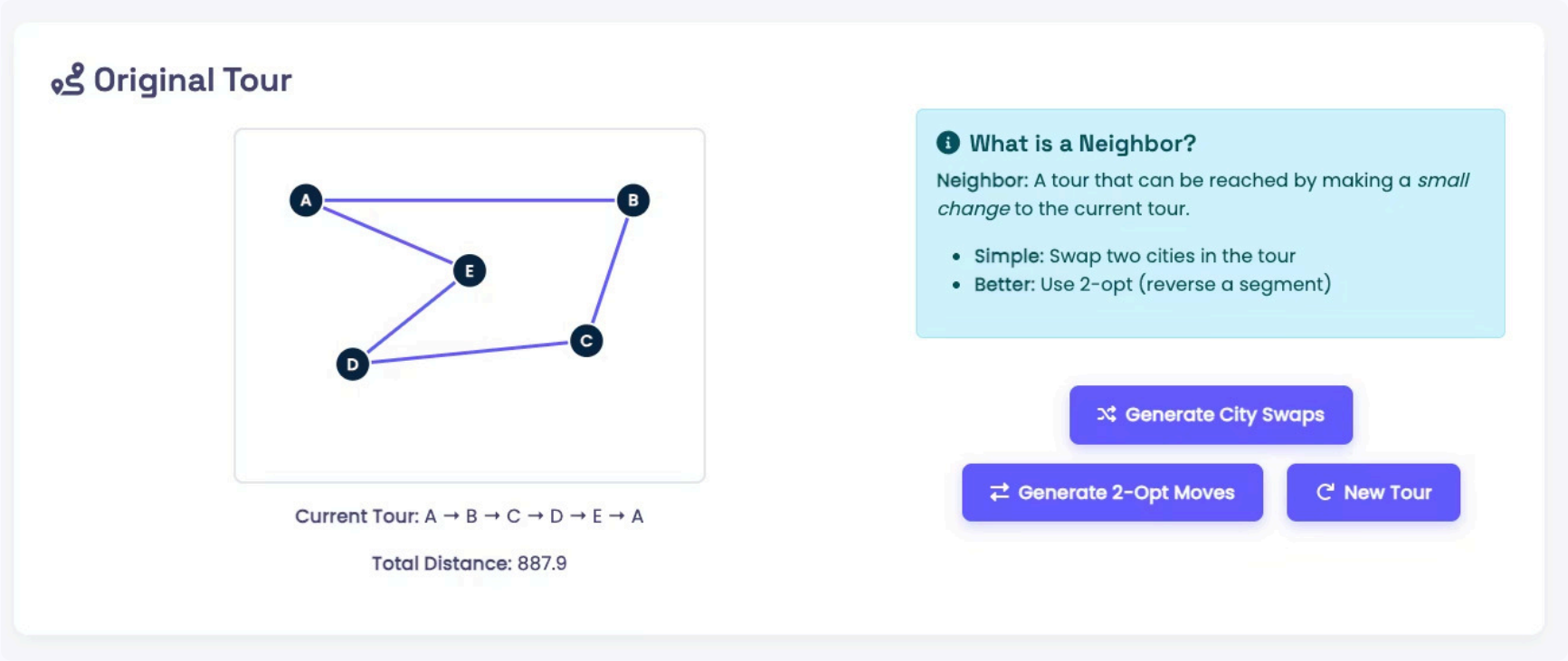


Reversing segment C→E→D to D→E→C

State representation changes:

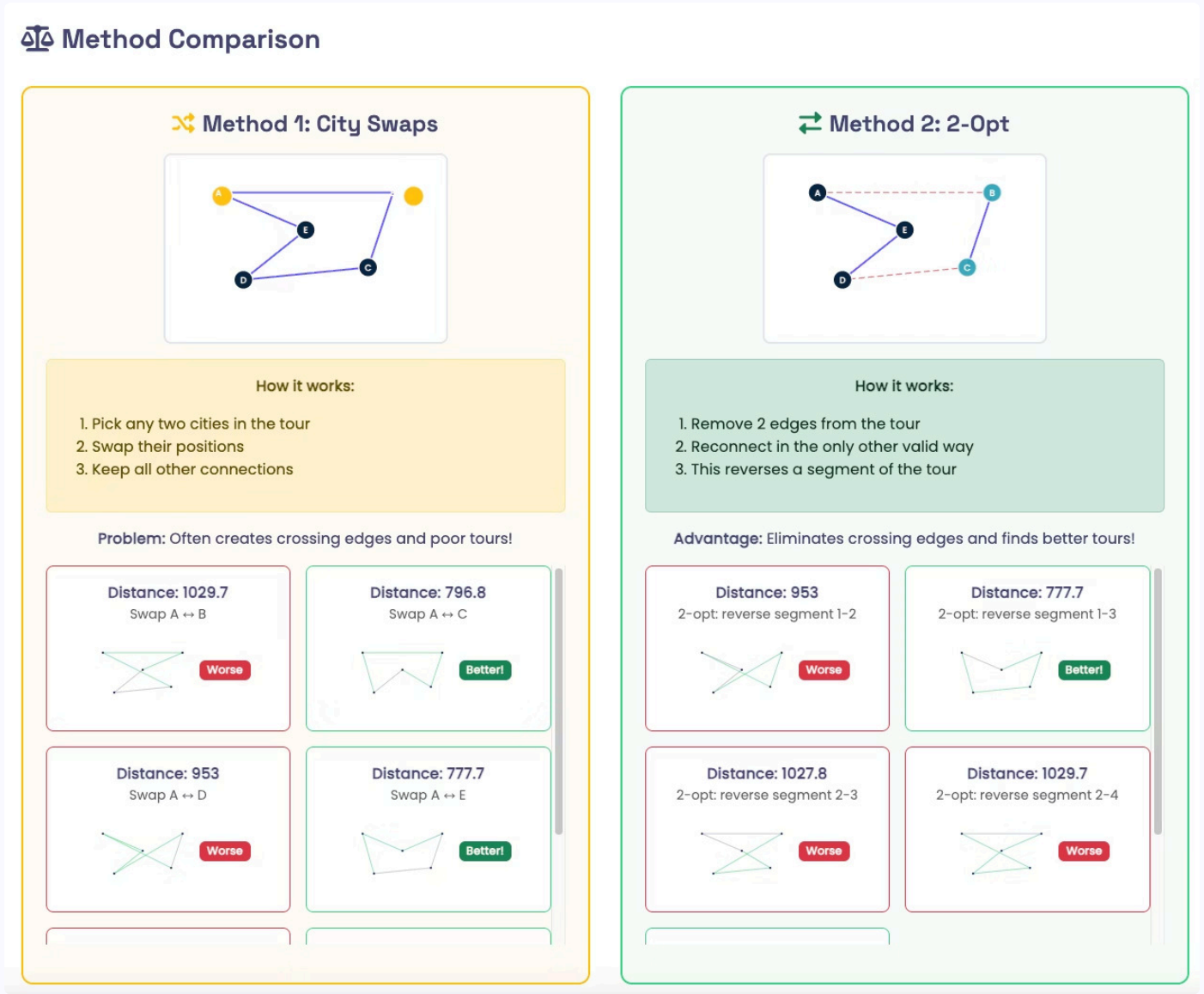
- Before: **ABCED**
- After: **ABDEC**

Illustration: TSP Neighbor Generation



[Hill Climbing TSP - Complete Algorithm Execution]

Illustration: TSP Neighbor Generation



Hill-Climbing Search Variants

Hill-climbing search typically starts with **a randomly generated state** and then **makes incremental improvements** by moving to better neighboring states.

Steepest-Ascent Hill Climbing

- **Examines all possible** successor states
- Chooses the successor with the **highest utility value**
- Guarantees the **best immediate improvement**
- More computationally expensive per step

Stochastic Hill Climbing

- **Chooses randomly** among uphill moves
- **Probability of selection** proportional to improvement
- Can escape some plateaus and small local optima
- May not take the steepest path

First-Choice Hill Climbing

- **Generates random successors** until a better one is found
- Most popular variant in practice
- Efficient for large branching factors
- Avoids evaluating all neighbors

Each variant offers different trade-offs between computational efficiency and solution quality, making them suitable for different types of problems.

Local Optima: Hill-Climbing's Limitation

⚠️ The Local Optima Problem in Hill Climbing

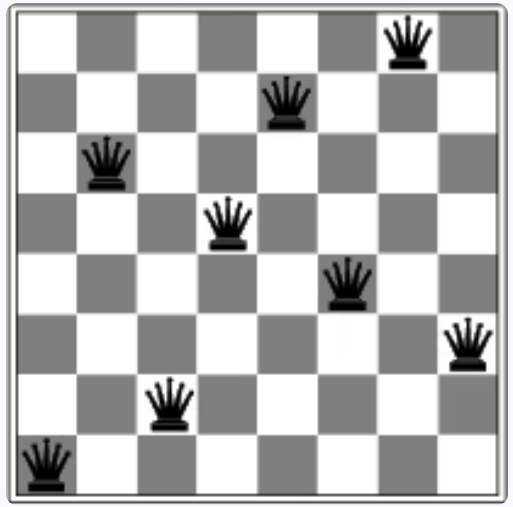
🔹 Key Limitations

- ❌ **Not complete** → may get stuck in a local maximum
- ❌ **Not optimal** → solution might not be the global maximum
- ⛰️ **Plateaus** → flat regions with no improving moves
- ↔️ **Ridges** → oscillation between equally good states

💡 Simple Fix: Random-Restart Hill Climbing

- 🔄 Run hill climbing **many times** with different random starting states
- 🏆 Keep the **best solution** found across runs
- 🎲 Increases chance of reaching the **global maximum**

👉 **Takeaway:** Hill climbing is fast and simple but easily trapped; **random restarts** make it much more powerful.



Local optimum in 8-queens problem

In this configuration, no single queen can be moved within its column to reduce the number of conflicts. The algorithm is stuck despite not having found a global optimum (zero conflicts).

The State Space "Landscape"

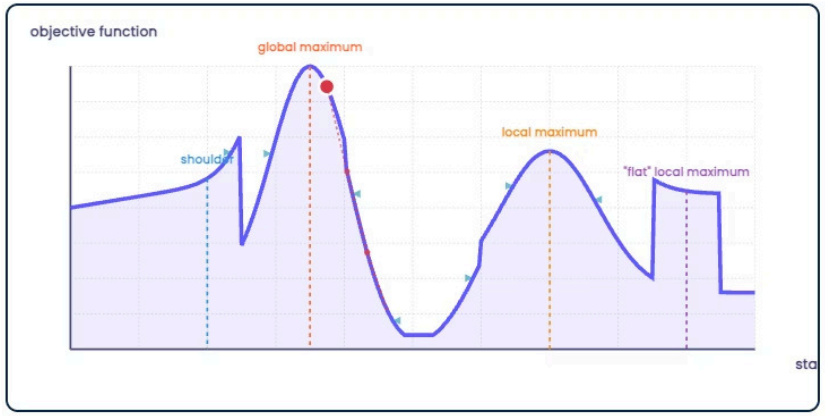
🔹 How to Visualize It

- 🏔️ **Height = utility value**
- 📍 **Neighbors = nearby states**
- 🧗 **Hill-climbing = climbing uphill** toward the highest point

🔹 Common Features

- 🔺 **Local maxima** → higher than neighbors, but not the best
- 🏆 **Global maximum** → the highest point overall
- ➖ **Plateaus** → flat areas with no improving moves
- 🪜 **Shoulders** → flat areas that eventually lead upward
- ↔️ **Ridges** → narrow paths where progress is tricky

👉 **Key Insight:** The search space can trap hill-climbing unless strategies (like random restarts or simulated annealing) help escape.



Understanding the shape of the utility landscape helps us choose appropriate search algorithms and diagnose problems when search gets stuck.

aniskoubaa.org

1D Search Landscape – Hill Climbing Visualization

Click any scenario to see hill climbing behavior at key landscape features:

Solutions to Landscape Challenges:

- 🔺 **Local maxima** → use **random restarts** (try from different initial states)
- ➖ **Plateaus** → allow **sideways moves** (accept equal-value states)
- ↔️ **Ridges** → use **momentum or smarter moves** to avoid oscillation
- 🌋 **Complex landscapes** → apply **advanced methods** like **simulated annealing**

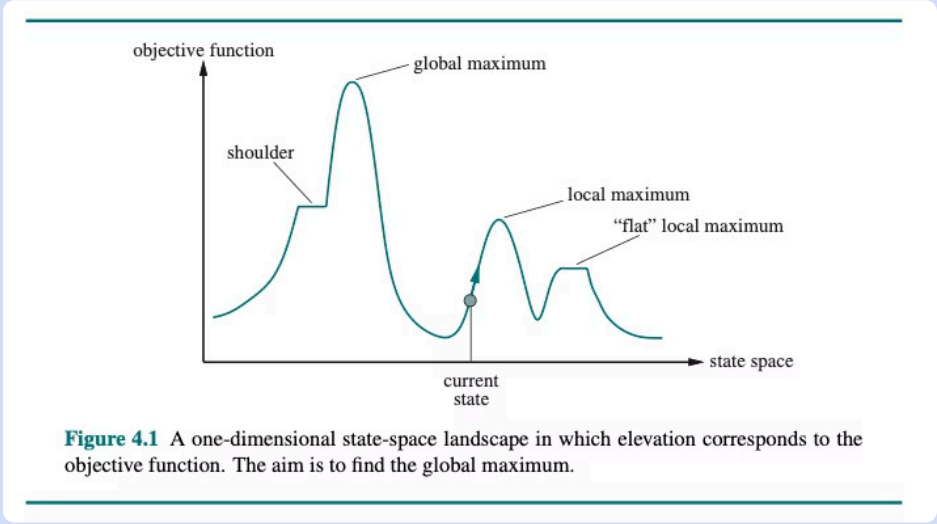


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum.



Minimization vs. Maximization

Optimization problems involve finding the best solution, whether by maximizing a desired outcome or minimizing an undesirable one.

Maximization:

- **Goal:** Find the highest point
- **Example:** Maximize profit
- **Approach:** Move towards increasing utility (e.g., Hill-climbing)

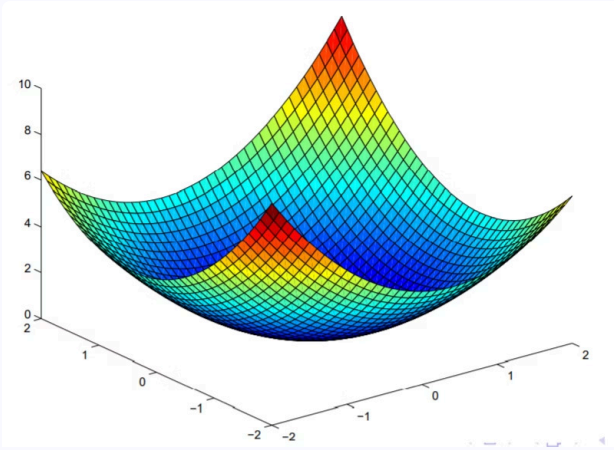
Minimization:

- **Goal:** Find the lowest point
- **Example:** Minimize cost
- **Approach:** Move towards decreasing cost (e.g., Gradient Descent)

- 🎯 Any **maximization** problem can be turned into a **minimization** problem (and vice versa) by **negating the objective function**.
- 🔁 The algorithms stay essentially the **same** – only the **direction of improvement** changes.

Convex vs. Non-Convex Optimization

Convex Optimization:

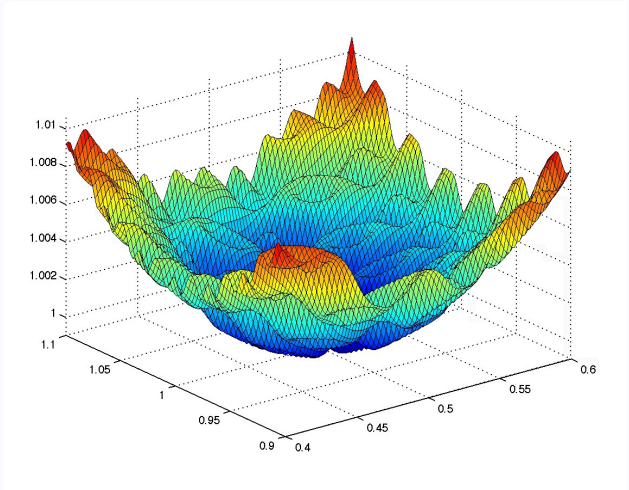


Characteristics:

- One global optimum
- Smooth, bowl-shaped function
- Local optimum is always global
- Any descent method will find the global optimum
- Efficient analytical methods available

Examples: Linear programming, least squares regression, support vector machines

Non-Convex Optimization:



Characteristics:

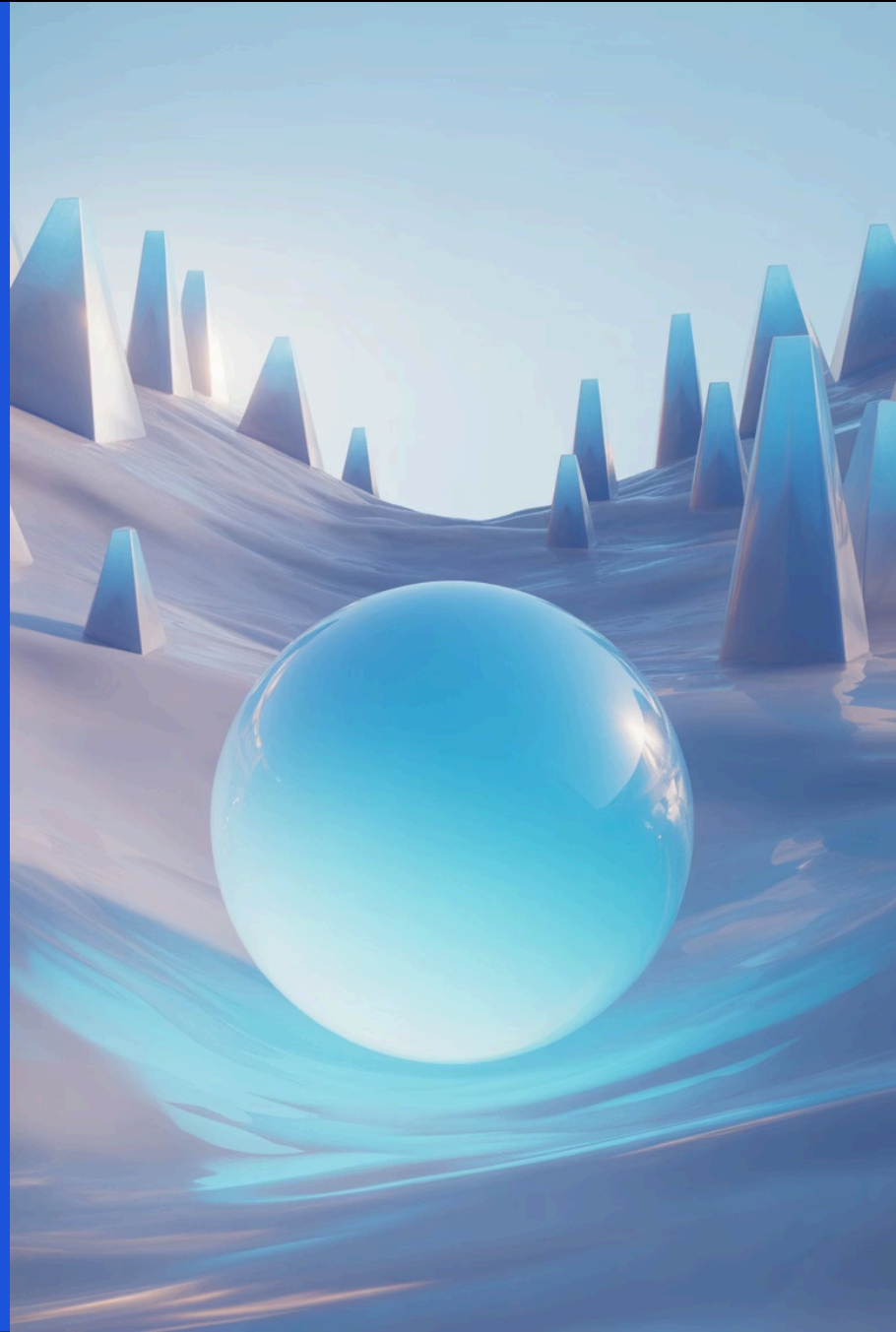
- Multiple local optima
- Complex, rugged landscapes
- Local methods can get stuck
- Finding global optimum is challenging
- Requires more sophisticated approaches

Examples: Neural network training, TSP, n-Queens, protein folding

Most discrete optimization problems in AI are non-convex, making them algorithmically challenging. This is why techniques like simulated annealing and genetic algorithms are particularly valuable.

Simulated Annealing

A probabilistic technique for approximating the global optimum of a given function, inspired by the process of annealing in metallurgy.



Simulated Annealing

🔥 Core Idea

- Combines **stochastic hill climbing** with the ability to **escape local minima**
- Occasionally accepts “bad” moves to explore beyond the neighborhood

🔹 Key Features

- 🏭 Inspired by **physical annealing** in metallurgy
- 🌡️ Uses a **temperature parameter** to balance:
  - **Exploration** (high temp → accept worse states often)
  - **Exploitation** (low temp → accept worse states rarely)
- 📉 **Cooling schedule**: temperature gradually decreases over time

👉 **Takeaway:** Simulated annealing avoids getting stuck by allowing controlled randomness, making it more likely to find the **global optimum**.

Acceptance Probability

For a **minimization problem**, the **probability** of accepting a move to a worse state is:

$$P(accept) = e^{-\frac{\Delta E}{T}}$$

Where:

- **ΔE** = VALUE(next) - VALUE(current)
- T = current temperature (control the randomness)

Role of Temperature

- **High temperature:**
  - More willing to accept “bad moves.”
  - Promotes **exploration** (searching widely).
- **Low temperature:**
  - Rarely accept worse moves.
  - Promotes **exploitation** (refining the best area).
- Gradually **cooling down** balances exploration → exploitation.

🔥 Physical Annealing in Metallurgy

The concept of simulated annealing in AI is directly inspired by a physical process in metallurgy called annealing, used to strengthen and stabilize metals.

The Physical Process:

- **Heat the Metal:** Raise temperature until atoms move freely and randomly.
- **Slow Cooling:** Gradually reduce the temperature, allowing atoms sufficient time to explore configurations.
- **Stable Structure:** Atoms settle into a low-energy, highly ordered, stable crystal lattice.

Why it matters: Rapid cooling "freezes" atoms in a disordered, unstable state (like a local minimum), while slow cooling enables them to find the globally optimal, lowest energy configuration.

👉 **Takeaway:** Simulated annealing uses controlled randomness, allowing it to explore broadly at first (high temp) and then focus on refining the best solutions (low temp), mirroring the physical annealing process to find a global optimum.

🔗 Connection to AI:

"Atoms" = States

In AI, these are the possible configurations or solutions in the search space.

"Energy" = Cost Function

The "energy" of the metal corresponds to the cost or objective function value in an AI problem.

"Temperature" = Randomness

This parameter controls the probability of accepting worse solutions; high temperature allows more exploration.

"Slow Cooling" = Cooling Schedule

Gradually reducing the "temperature" helps the algorithm transition from exploration to exploitation, settling on good solutions.

Simulated Annealing Algorithm

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current ← problem.INITIAL
  for t = 1 to ∞ do
      T ← schedule(t)
      if T = 0 then return current
      next ← a randomly selected successor of current
      ΔE ← VALUE(current) – VALUE(next)
      if ΔE > 0 then current ← next
      else current ← next only with probability e^(ΔE/T)
```

Figure 4.5 The simulated annealing algorithm, a version of stochastic hill climbing where some downhill moves are allowed. The *schedule* input determines the value of the "temperature" T as a function of time.

🔥 Simulated Annealing Algorithm

1️⃣ Initialize

- Start with a **random initial state**.
- Set a **high initial temperature (T)**.

2️⃣ Generate Neighbor

- Randomly pick a **neighboring state** of the current state.

3️⃣ Evaluate Move

- Compute: ΔE = VALUE(next) - VALUE(current)
- If **ΔE < 0** (better state): ✅ Always accept the move.
- If **ΔE > 0** (worse state): 🎲 Accept with probability: $P = e - \Delta E/TP = e^{-\Delta E/T}$P=e−ΔE/T

4️⃣ Cool System

- Reduce temperature gradually using a **cooling schedule** (e.g., $T \leftarrow 0.95 \times TT \leftarrow 0.95 \times T$T←0.95×T).

5️⃣ Terminate

- Stop when:
  - Temperature reaches a **minimum threshold**, OR
  - No improvement after several iterations.

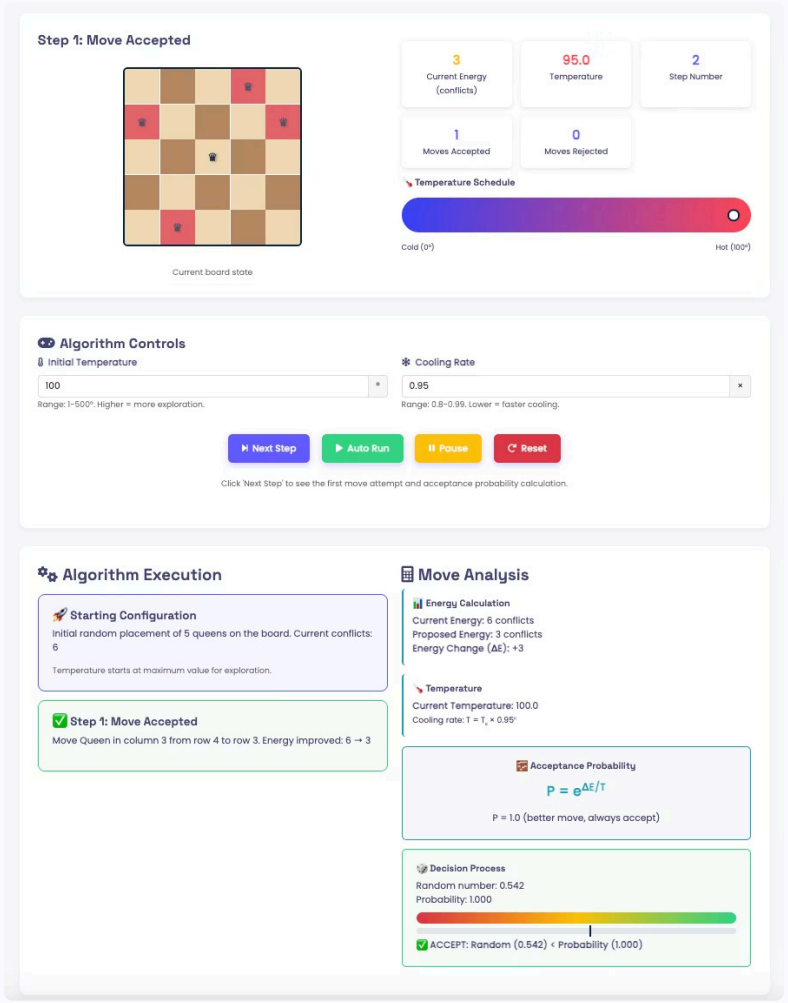
Pseudocode:

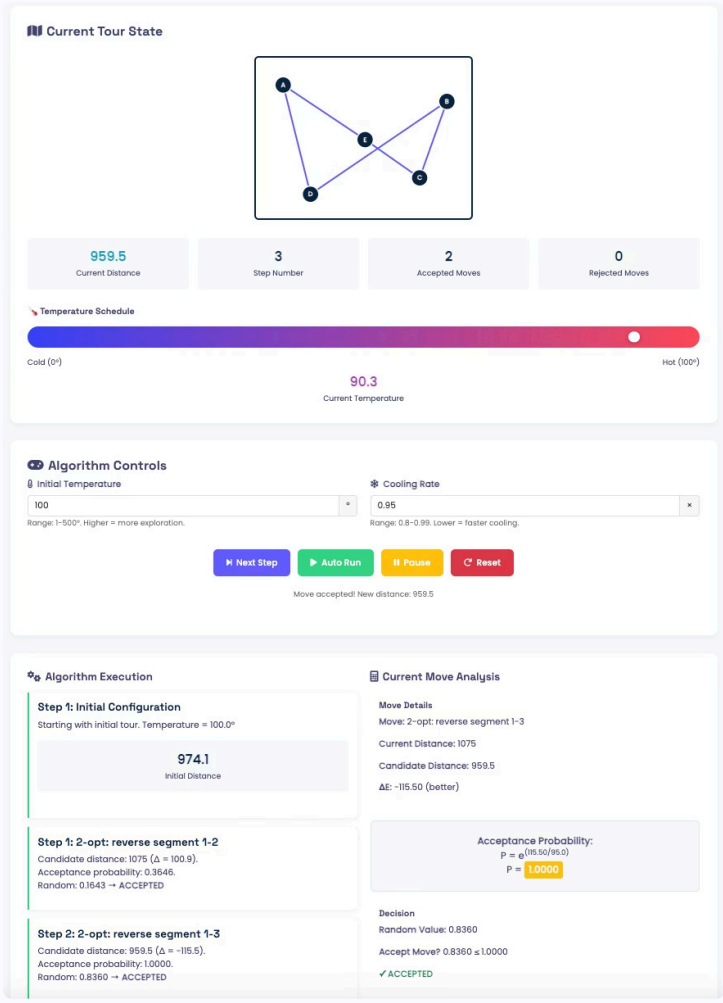
```
function SIMULATED_ANNEALING(initial_state):
    current_state = initial_state
    current_energy = COST(current_state)
    temperature = INITIAL_TEMPERATURE
    while temperature > MIN_TEMPERATURE and not STOP_CRITERION_MET:
        neighbor_state = GENERATE_NEIGHBOR(current_state)
        neighbor_energy = COST(neighbor_state)

        delta_E = neighbor_energy - current_energy

        if delta_E < 0:  // Better state (minimization)
            current_state = neighbor_state
            current_energy = neighbor_energy
        else:  // Worse state
            acceptance_probability = EXP(-delta_E / temperature)
            if RANDOM_0_1() < acceptance_probability:
                current_state = neighbor_state
                current_energy = neighbor_energy
        temperature = COOLING_SCHEDULE(temperature)
    return current_state
```

Simulated Annealing Demos





Simulated Annealing: What It Promises vs. Reality

If the "temperature" decreases **slowly enough**, Simulated Annealing can theoretically find the absolute best solution.

The Problem:

This requires an i**mpossibly slow logarithmic decrease**, as shown by:

$$T(t) = \frac{T_0}{\log(1+t)}$$

Achieving this theoretical guarantee means running the algorithm for an impractical amount of time, making it unfeasible for most real-world problems.

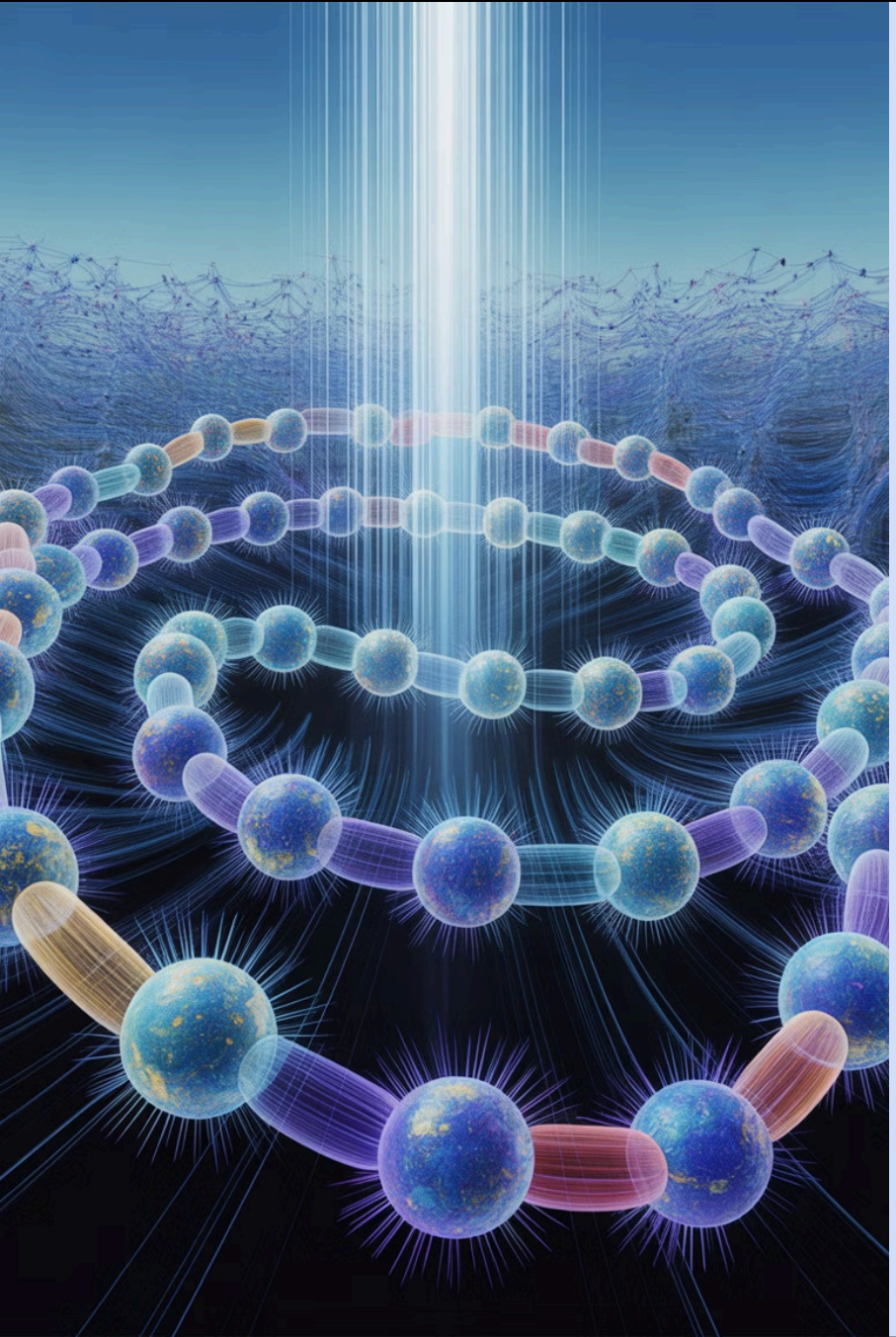
How We Actually Do It:

In practice, we use **faster cooling schedules**. Common types include:

- **Exponential:** Temperature drops by a fixed percentage each step.
- **Linear:** Temperature drops by a fixed amount each step.
- **Adaptive:** Cooling rate adjusts based on solution acceptance.

These methods provide very good solutions much quicker, balancing solution quality with computational efficiency.

👉 **Simulated Annealing is about finding a balance: we want a great solution, but we also need the algorithm to finish quickly. In practice, we accept faster runs that give very good (even if not perfect) results.**



Evolutionary Algorithms

A Population-based Metaheuristic

Evolutionary algorithms mimic the process of natural selection to solve complex optimization problems by evolving a population of candidate solutions over multiple generations.

Evolutionary Algorithms: Key Concepts

Core Principles:

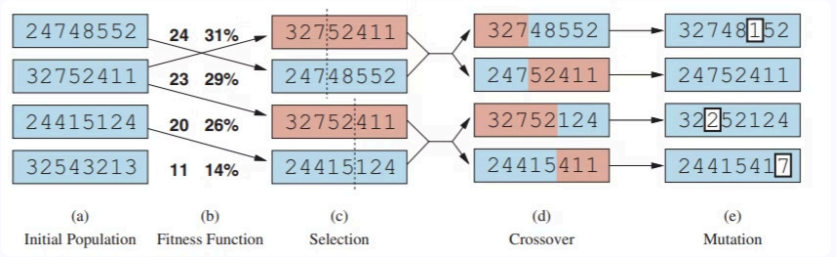
Evolutionary algorithms are a family of population-based optimization methods inspired by biological evolution, including:

- Genetic Algorithms (GAs)
- Evolutionary Strategies (ES)
- Genetic Programming (GP)
- Evolutionary Programming (EP)

Key Mechanisms:

- **Reproduction:** Selection based on fitness function
- **Crossover:** Combining information from multiple parents
- **Mutation:** Random variation to maintain diversity
- **Survival:** Competition for limited resources

8-Queens Example:



Chromosome representation: Array with row position of queen in each column

Example chromosomes: [2,4,6,8,3,1,7,5], [3,1,7,5,8,2,4,6]

Fitness function: Number of non-attacking queen pairs (max 28 for 8-queens)

Crossover: Split and combine chromosomes at random point

Mutation: Randomly change queen position in one column

Algorithm Flow:

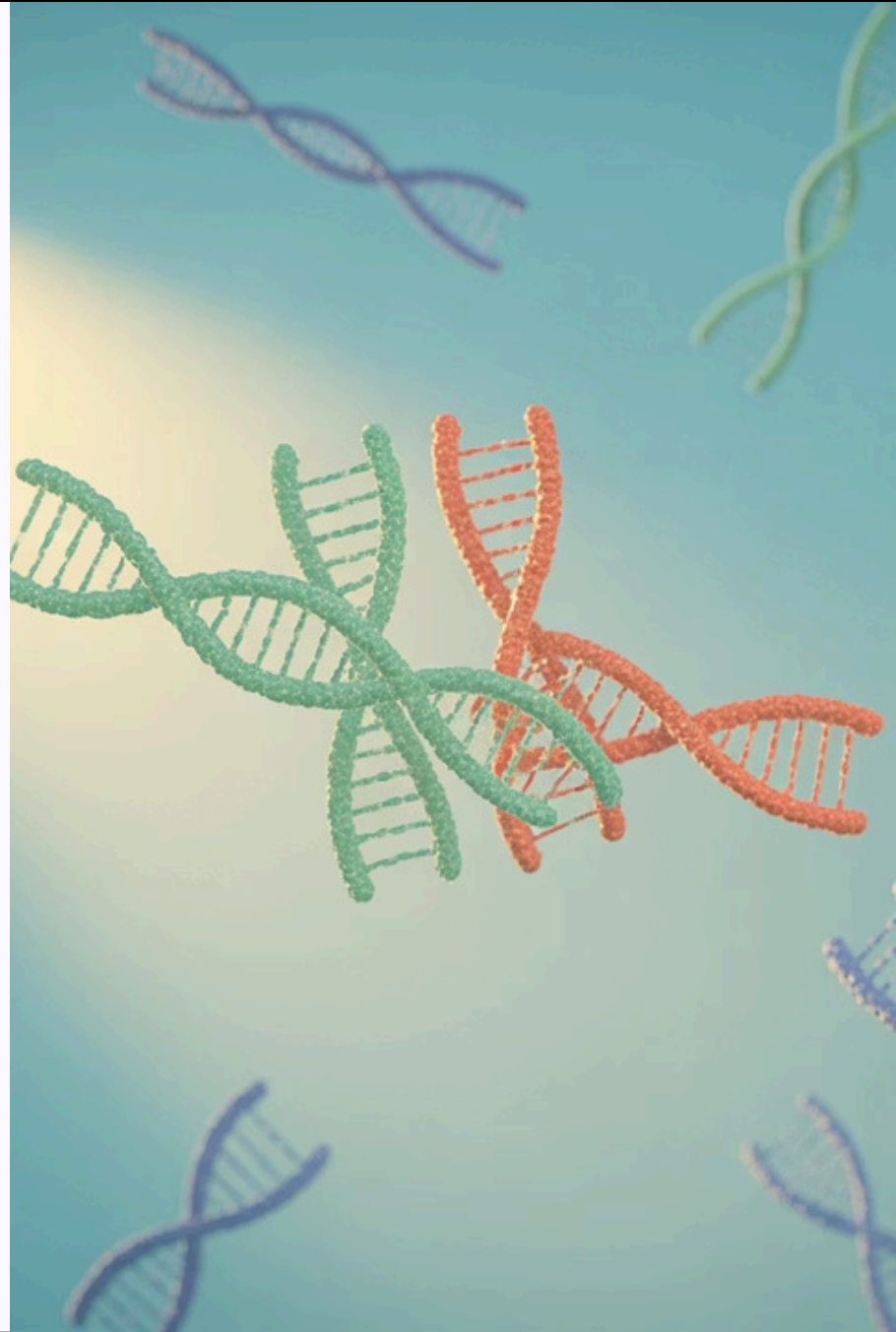
Initialize random population → Evaluate fitness → Select parents → Reproduce with crossover and mutation → Evaluate offspring → Form next generation → Repeat until satisfactory solution found

🧬 Introducing Genetic Algorithms

Core Idea:

- Inspired by **natural evolution**
- Uses a **population of solutions** instead of a single one
- Improves solutions over generations using:
  - **Selection** → survival of the fittest
  - **Crossover** → combine traits of parents
  - **Mutation** → introduce random variation

👉 **GA = Search + Optimization by Evolution**



♟️ n-Queens as the Example Problem

Problem: Place n queens on an n×n chessboard so that no two queens attack each other.

Representation (chromosome):

One queen per column → represent solution as an array of length n.

→ ✅ Compact encoding

→ ✅ Automatically avoids column conflicts

Example (4-Queens):

```
[2, 4, 1, 3]
```

Means:

- $Col_1 \rightarrow Row_2$
- $Col_2 \rightarrow Row_4$
- $Col_3 \rightarrow Row_1$
- $Col_4 \rightarrow Row_3$

🧮 Fitness Function

How good is a solution?

Defining Fitness:

Fitness = number of non-attacking pairs of queens

Max fitness = total pairs =

$$n(n-1)/2$$

Or equivalently:

Fitness = – (number of conflicts) (maximize fewer conflicts)

Example:

[2, 4, 1, 3] → 0 conflicts → fitness = max (optimal)

[1, 1, 1, 1] → many conflicts → very low fitness

👫 Selection (Choosing Parents)

Goal: To prefer solutions with higher fitness, driving the population towards optimal outcomes while maintaining diversity.

🎡 Roulette Wheel Selection

Solutions are chosen with a **probability proportional to their fitness score**. Fitter individuals get larger "slices" of the wheel, increasing their chances of being selected.

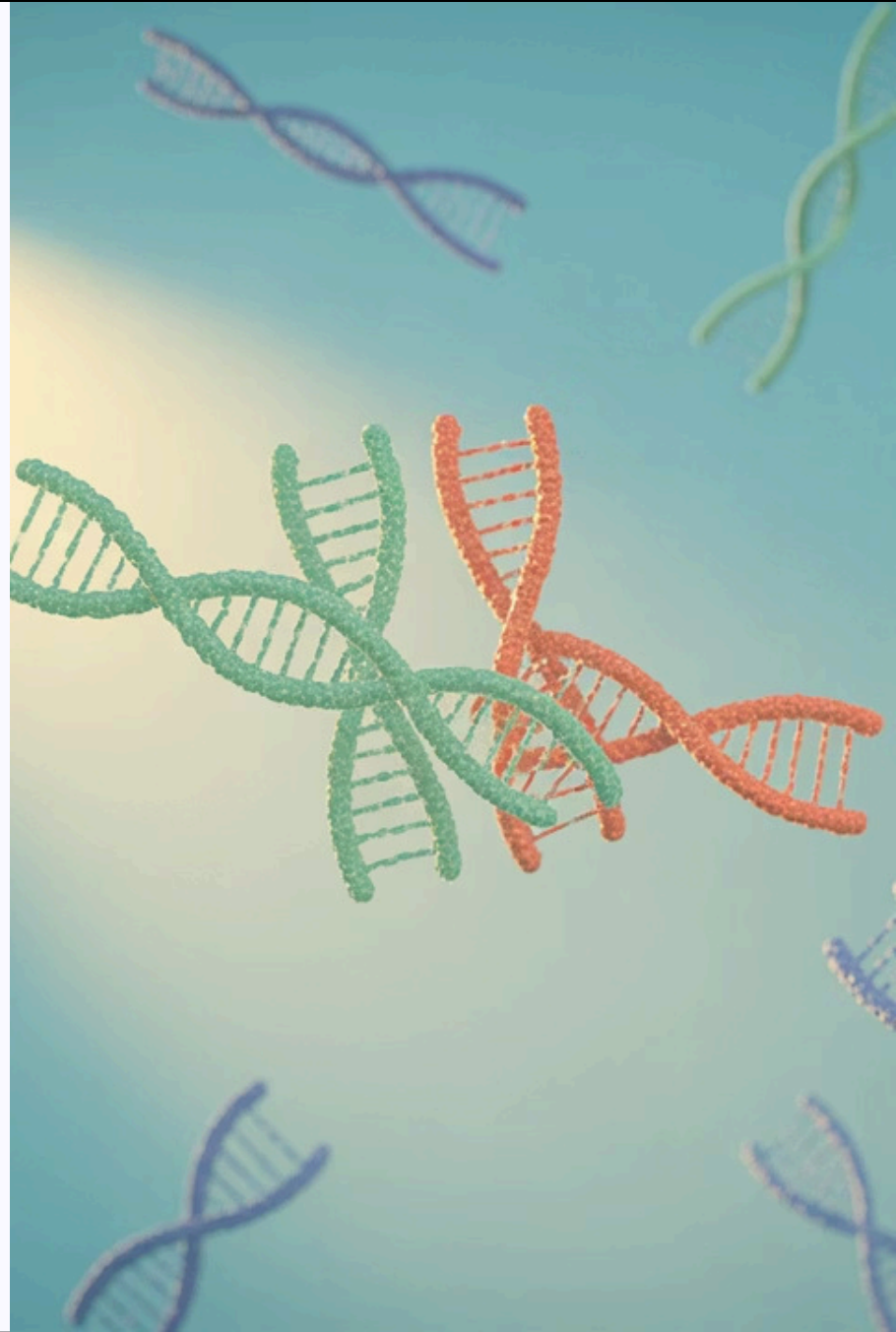
🏆 Tournament Selection

A small group of individuals is randomly picked from the population. The fittest individual within this group is then selected to be a parent.

📊 Rank Selection

Individuals are chosen based on their rank (from fittest to weakest) rather than their raw fitness values. This can help prevent premature convergence if a few individuals are exceptionally fit.

✨ This ensures stronger solutions are more likely to pass on their traits, but weaker ones still get a chance, which is vital for exploring new possibilities.



👫 Selection (Choosing Parents)

Goal: Prefer solutions with higher fitness

Roulette Wheel

chance proportional to fitness

Tournament

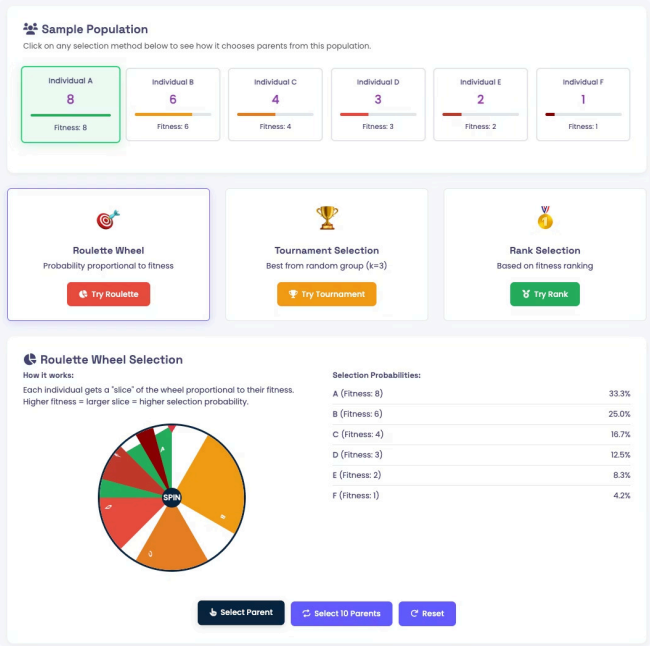
pick best out of k random individuals

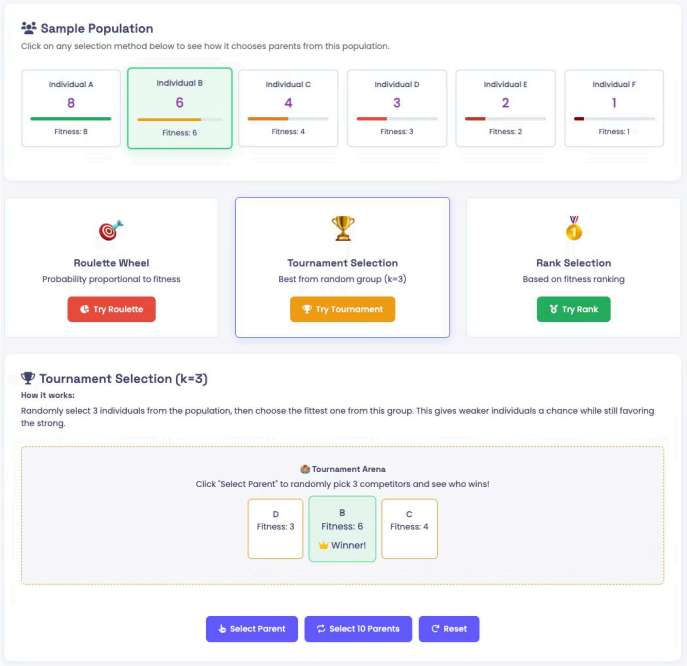
Rank Selection

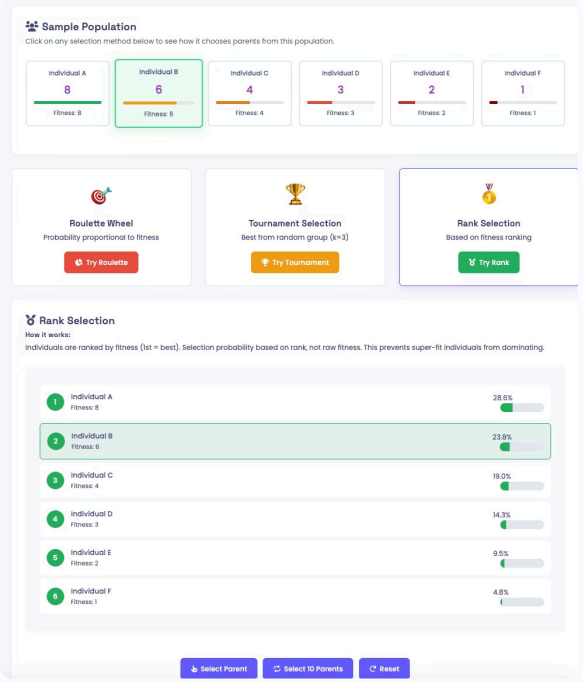
rank solutions and assign probabilities

👉 Stronger solutions more likely to be chosen, but weaker ones still have a chance (maintains diversity).

👫 Selection (Choosing Parents)







🔀 Crossover (Recombination)

Crossover is a key genetic operator that combines genetic material from **two parent solutions** to create **new offspring (children)**. This process helps explore the search space by mixing existing traits and generating novel combinations, driving the population towards better solutions.

One-Point Crossover Example:

Using the n-Queens representation, we select a crossover point and swap genetic material beyond that point from two parents:

```
Parent A: [2, 4, 1, 3]
Parent B: [3, 1, 4, 2]

Crossover point (after 2nd gene):

Child:    [2, 4 | 4, 2]
```

⚠ ⚠️ Important Consideration:

Crossover can sometimes produce **duplicate values** or **invalid solutions**. **Repair mechanisms** or specific crossover operators (like Partially Matched Crossover for permutations) are often used to ensure valid and meaningful offspring.

🎲 Mutation

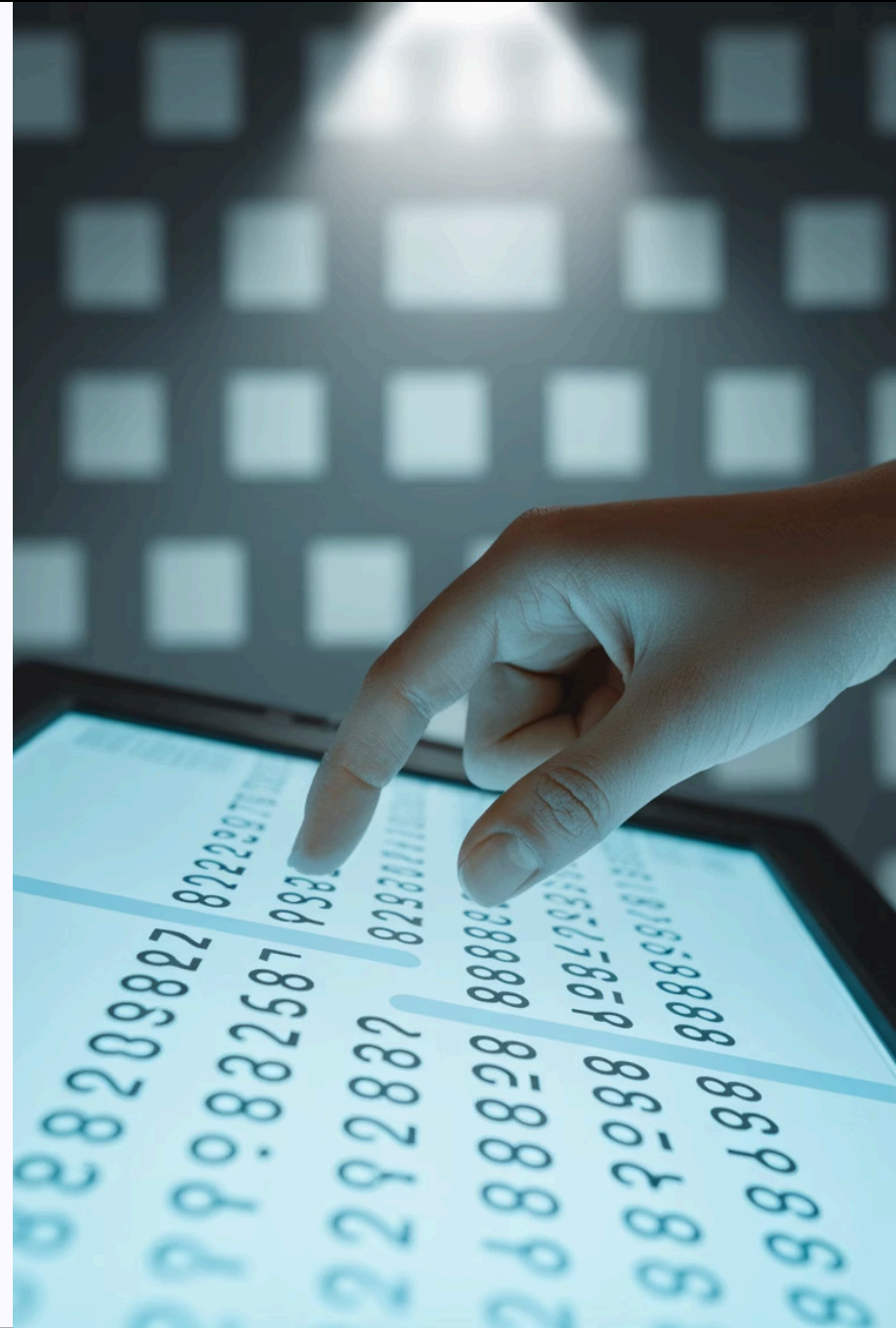
Mutation is a crucial genetic operator that introduces small, random changes into an individual solution (chromosome). Its primary roles are:

- To maintain **genetic diversity** across generations.
- To prevent the algorithm from getting stuck in **local optima**.
- To explore new regions of the search space that might not be accessible through crossover alone.

In the context of the n-Queens problem, a mutation operation might involve randomly changing the row position of a single queen in one of the columns:

```
Chromosome Before: [2, 4, 4, 2]  (Example for 4-Queens)
Mutation applied:   Changing the 2nd queen's row from 4 to 3
Chromosome After:  [2, 3, 4, 2]
```

👉 This random adjustment helps the algorithm discover entirely new configurations, ensuring a more thorough search for optimal solutions.



🔁 GA Iterative Process

1️⃣ Start with a **random population** of solutions
2️⃣ Evaluate fitness of all solutions
3️⃣ **Select parents**
4️⃣ Apply **crossover & mutation** to produce children
5️⃣ Form new population (next generation)
6️⃣ Repeat until:

- Found solution with o conflicts
- Or max generations reached

🔁 GA Iterative Process

Initialize Population

Start with a diverse set of random solutions.

Evaluate Fitness

Assess each solution's quality with the fitness function.



Termination

Repeat until optimal solution or max generations reached.

Select Parents

Choose fitter individuals for reproduction.

Form New Generation

Replace the current population with the new set of solutions.

Crossover & Mutation

Generate new offspring by combining and altering genes.

This iterative process continuously refines the population, gradually improving the solutions over successive generations until a satisfactory outcome is achieved or a predefined limit is met.

📈 GA on n-Queens (Illustrated Example)

Let's trace a simplified journey of a Genetic Algorithm finding a solution to the n-Queens problem.

01

Generation 0 (Random start)

A population of randomly generated candidate solutions:

```
[1, 3, 4, 2]
[4, 1, 3, 2]
[2, 2, 4, 1]
...
```

02

Evaluate Fitness

Each chromosome's fitness is assessed by counting the number of non-attacking queen pairs (or conflicts). Solutions with fewer conflicts have higher fitness.

03

Select Best Parents

Based on their fitness scores, individuals are chosen to become parents for the next generation. Higher fitness increases the probability of selection.

04

Apply Crossover

Genetic material is exchanged between selected parents to create offspring. For example, combining two parents:

Parent A: `[1, 3, 4, 2]`

Parent B: `[4, 1, 3, 2]`

could result in an offspring like: `[1, 3, 3, 2]`

05

Apply Mutation

Random changes are introduced to the offspring's genetic material to maintain diversity and explore new solution spaces.

For example, mutating `[1, 3, 3, 2]` might change the third gene, resulting in: `[1, 3, 2, 2]`

06

Iterate Towards Optimum

This cycle of evaluation, selection, crossover, and mutation repeats over many generations.

Over generations, conflicts decrease, eventually leading to the optimal solution:

```
[2, 4, 1, 3]
```

🧬 GA vs. Other Local Search Methods

Comparison of Genetic Algorithms with Hill Climbing and Simulated Annealing:

Population vs. Single Solution	GA: Works with multiple solutions simultaneously	Hill Climbing & SA: Work with one solution at a time
Exploration Strategy	GA: Combines exploration (crossover/mutation) with exploitation (selection)	Hill Climbing: Pure exploitation (greedy) SA: Controlled exploration via temperature
Memory Requirements	GA: Higher (stores entire population)	Hill Climbing & SA: Lower (single solution)
Parallelization	GA: Naturally parallel (population-based)	Hill Climbing & SA: Sequential by nature